inmind.academy
by inmind.ai

From Prototype to Production

Cloud, APIs, Optimization, Architecture, Lessons Learned

SPRING 2026

Today's Roadmap (Extended Session)

This is our final technical session. Everything you need to go from a working notebook to a production-ready system.

- **Part 1:** Cloud AI Platforms
- **Part 2:** Managed RAG Services
- **Part 3:** Building Production APIs
- **Part 4:** Inference Optimization
- **Part 5:** Hardware Fundamentals
- **Part 6:** Fine-Tuning Decision Framework
- **Part 7:** Security, Cost, Scalability
- **Part 8:** Architecture Patterns, Pitfalls, Lessons

The Full Stack You Have Built

- **Sessions 4-6:** Classical NLP, Deep Learning, Transformers (the foundations)
- **Session 7:** LLM usage: local (Ollama) + API, prompting techniques, LangChain
- **Session 8:** RAG: chunking, embeddings, vector databases, retrieval, generation
- **Session 9:** Agents: reasoning patterns, tool calling, LangGraph
- **Session 10:** Multi-agent systems, MCP, guardrails, human-in-the-loop
- **Session 11:** Multimodal pipelines (voice, vision), evaluation (RAGAS, LLM-as-judge), observability
- **Today:** How to take all of this and build something that works in the real world, at scale, securely, cost-effectively.

Part 1: Cloud AI Platforms

AWS Bedrock, Azure AI, GCP Vertex AI. What each offers, how they compare, and when to use which.

Why Use a Cloud AI Platform?

You have been running locally (Ollama/vLLM) and calling APIs directly. This works for prototyping. Production needs more:

- **Reliability:** 99.9%+ uptime SLAs. Your local machine going to sleep is not acceptable.
- **Scalability:** Handle 10 users or 10,000 users without changing code. Auto-scale up during peak, down to save cost.
- **Security:** Enterprise-grade authentication, encryption, audit logging, compliance certifications (SOC 2, HIPAA, GDPR).
- **Managed infrastructure:** Someone else handles GPU provisioning, model serving, load balancing, updates.
- **Integration:** Connect to existing enterprise systems (databases, identity providers, storage, monitoring).
- **Model marketplace:** Access many models (open + proprietary) through a single API with unified billing.

Your choice often depends on which cloud the company already uses, not which AI platform is technically "best."

AWS Bedrock

- **Models:** Claude (Anthropic), LLaMA 3.1 (Meta), Mistral, Amazon Titan, Cohere, AI21, Stability AI
- **Knowledge Bases:** Managed RAG. S3 -> chunk -> embed -> OpenSearch Serverless or Pinecone. Query via API.
- **Agents:** Built-in ReAct loop. Tools via Lambda. Knowledge base integration.
- **Guardrails:** Content filtering, PII redaction, topic blocking. Declarative policies.
- **Fine-tuning:** Custom training for Titan, LLaMA, Cohere.
- **Model evaluation:** Built-in tools for comparing models on your data.
- **Pricing:** Pay-per-token (on-demand) or provisioned throughput (reserved).

Reference: https://aws.amazon.com/bedrock/

Best when: The organization is already on AWS. Bedrock integrates natively with S3, Lambda, IAM, CloudWatch, and the rest of the AWS ecosystem. Broadest model selection from multiple providers.

Azure AI Foundry + Azure OpenAI Service

- **Azure OpenAI Service:** Exclusive access to GPT-4o, GPT-5, o1, DALL-E 3, Whisper on Azure infrastructure. Same OpenAI API with Azure security. Your data NOT used for training.
- **AI Foundry:** Broader catalog: OpenAI + LLaMA + Mistral + Phi + Cohere. Prompt flow (visual workflow). Model benchmarking. Responsible AI dashboard.
- **Azure AI Search:** Vector + keyword hybrid search. Semantic ranker (built-in re-ranking). Best managed RAG retrieval. Used by Microsoft Copilot internally.
- **Document Intelligence:** PDF, image, table, form parsing. OCR built in. Integrated with AI Search indexing.
- **Security:** Entra ID, VNet, Private Endpoints. Data stays in your tenant.

Reference: https://azure.microsoft.com/en-us/products/ai-services/openai-service, https://ai.azure.com/

Best when: You need GPT-4o specifically, or your organization is on Azure/Microsoft 365.

Google Cloud Vertex AI

- **Models:** Gemini 3.5 Pro/Flash, Gemini 3.0 Flash. Open models via Model Garden (150+). Claude available via partnership.
- **Vertex AI Search:** Managed RAG. Document parsing, chunking, retrieval. Grounded generation with citations.
- **Grounding:** UNIQUE: ground responses in Google Search results AND/OR your own data. Real-time knowledge.
- **Agent Builder:** Build agents with tools, RAG, Dialogflow CX integration.
- **Tuning:** Supervised fine-tuning, RLHF, distillation for Gemini.
- **Model Garden:** 150+ models deployable with one click.
- **Multimodal:** Best native multimodal (text + image + audio + video in Gemini).

Reference: https://cloud.google.com/vertex-ai

Best when: You need Gemini, Google Search grounding, multimodal natively, or your organization is in the Google ecosystem.

Cloud AI Platforms: Head-to-Head

Feature	AWS Bedrock	Azure AI	GCP Vertex AI
Proprietary models	Claude, Titan	GPT-4o, o1 (exclusive)	Gemini (exclusive)
Open models	LLaMA, Mistral	LLaMA, Mistral, Phi	LLaMA, Mistral + 150 more
Managed RAG	Knowledge Bases (OpenSearch)	AI Search (hybrid + semantic)	Vertex AI Search
RAG quality	Good	Best (hybrid + re-ranker)	Very good (+ grounding)
Agents	Bedrock Agents (Lambda)	Microsoft Agent Framework	Agent Builder, Dialogflow
Fine-tuning	Select models	GPT-4o, select open	Gemini
Guardrails	Bedrock Guardrails	Content filtering	Responsible AI
Multimodal	Via Claude/Titan	GPT-4o vision	Gemini (native, best)
Grounding	Knowledge bases only	AI Search	Google Search + custom
Best when	Already on AWS	Need GPT-4o + enterprise	Need Gemini + Google ecosystem

How to Choose: Decision Framework

- **1. Existing cloud footprint (biggest factor):** If your company is on AWS, use Bedrock. Azure -> Azure AI. GCP -> Vertex. Migration costs are real. AI platform is almost never the reason to switch clouds.
- **2. Model requirements:** Need GPT-5.4? Azure (exclusive) or OpenAI directly. Gemini? Vertex. Claude? Bedrock or Vertex. Open models? Any of the three, or self-host.
- **3. RAG requirements:** Azure AI Search is the strongest managed RAG (hybrid + semantic re-ranking + document intelligence). If RAG quality is your top priority, Azure has an edge.
- **4. Regulatory requirements:** Healthcare, government, finance require specific certifications or data residency. All three offer compliance, but check specifics for your region and industry.
- **5. Budget:** Google AI Studio: generous free tier. Azure: $200 free credit. AWS: free tier for many services. At scale, negotiate enterprise agreements.

Part 2: Managed RAG Services

When to build your own RAG vs. use a managed service. Architecture, tradeoffs, real-world considerations.

Build Your Own RAG vs. Managed RAG

Challenge	DIY RAG (what you built)	Managed RAG
Document parsing	You handle (pdfplumber, Unstructured)	Built-in, often better
Chunking	You choose strategy + params	Auto-optimized or configurable
Embedding	You choose model, host it	Managed, auto-updated
Vector storage	ChromaDB/Qdrant (you manage)	Managed, auto-scaled
Hybrid search	You implement BM25 + vector	Built-in (especially Azure)
Re-ranking	You implement cross-encoder	Built-in (Azure semantic ranker)
Scaling	You handle infrastructure	Auto-scales
Updates	Re-index manually	Auto sync (S3/Blob/GCS)
Cost	Compute + storage + engineering time	Pay-per-query

The real tradeoff: DIY gives control and understanding. Managed gives speed and reliability. For learning: build yourself (which you did). For production: evaluate managed options seriously.

Azure AI Search: Deep Dive

```
Documents (Blob, SQL, SharePoint, custom)
    |-> [Indexer] -> [Skillset: OCR, entities, key phrases, custom]
    |-> [Index]: Vector fields + Text fields + Metadata
    |-> [Query]: Vector search + BM25 keyword + Semantic ranker
    |-> [Results] -> Your LLM for generation
```

- **Hybrid search:** Combines vector similarity (semantic) with BM25 keyword search. Catches exact names, codes, IDs that vector search misses.
- **Semantic ranker:** Microsoft-trained cross-encoder re-ranker. Runs after initial retrieval. Equivalent to re-ranking from Session 8, fully managed.
- **Integrated vectorization:** Built-in embedding via Azure OpenAI or open models. No separate pipeline needed.
- **AI enrichment:** OCR, entity extraction, language detection, PII detection, custom skills. Runs during indexing.
- **Incremental indexing:** Only re-processes changed documents. Critical for large, frequently updated knowledge bases.

AWS Bedrock Knowledge Bases

```
Documents in S3 -> [Auto-parse] -> [Chunk + Embed] -> [Vector Store]
                                                      (OpenSearch/Pinecone/Redis/Aurora)
Query -> [RetrieveAndGenerate API] -> One call: retrieve + generate
```

- **Setup:** Point to S3 bucket, Bedrock handles the rest. Configurable chunking (fixed, semantic, hierarchical).
- **Embedding:** Choice of model (Titan Embeddings, Cohere Embed). Vector store: managed OpenSearch Serverless or bring your own.
- **Features:** Metadata filtering, source attribution (citations), automatic sync on S3 changes.
- **Limitations vs Azure:** No hybrid search (vector only). No built-in re-ranker. Less advanced document parsing. Fewer enrichment options.

Reference: https://docs.aws.amazon.com/bedrock/latest/userguide/knowledge-base.html

GCP Vertex AI Search

- **Document processing:** Upload PDFs, web pages, structured data. Google handles parsing and chunking.
- **Grounded generation:** UNIQUE. LLM response grounded in your documents AND optionally Google Search results. Each claim has a citation.
- **Multi-turn conversation:** Built-in context tracking across turns without managing history yourself.
- **Blended search:** Vector + keyword, similar to Azure hybrid search.
- **Follow-up questions:** Auto-generates suggested follow-up questions for users.
- **Website search:** Can index and search an entire website, not just documents.
- **Unique advantage:** Google Search grounding: ground responses in real-time search index. For use cases needing current info (news, market data, regulations), extremely powerful.

Reference: https://cloud.google.com/generative-ai-app-builder

Managed RAG Services: Comparison

Capability	Azure AI Search	AWS Bedrock KB	GCP Vertex Search
Hybrid search	Yes (best-in-class)	Vector only	Blended search
Semantic re-ranking	Yes (built-in)	No	Limited
Document parsing	Excellent (Doc Intelligence)	Good (basic)	Good
OCR for scans	Yes (AI enrichment)	Basic	Yes
Table extraction	Yes	Limited	Yes
Web search grounding	No	No	Yes (unique)
Auto-sync on update	Yes	Yes (S3 events)	Yes
Metadata filters	Yes (OData)	Yes	Limited
Setup complexity	Medium	Low	Low
Best for	Complex enterprise RAG	Simple document Q&A	Google ecosystem + web

Part 3: Building Production APIs

From Jupyter notebook to a real API that frontends, mobile apps, and other services can call.

Why You Need an API

Your agent lives in a notebook. To make it usable, expose it as an API that other systems can call:

- Web frontend (React, Vue, Next.js)
- Mobile app (iOS, Android)
- Slack bot or Teams bot
- Another backend service (microservices)
- Webhook from third-party (CRM, helpdesk)

- **What the API does:**
- 1. Receives user message (HTTP POST + JSON)
- 2. Runs your agent pipeline
- 3. Returns response (JSON: answer + sources)
- 4. Optionally streams token by token

The standard tool: FastAPI. Most popular for AI/ML APIs. Fast, async-native (critical for LLM calls), auto-generated docs, native streaming support.

Reference: https://fastapi.tiangolo.com/

FastAPI: The Standard for AI APIs

```python
# pip install fastapi uvicorn
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI(title="TechNova Support Agent API")

class ChatRequest(BaseModel):
    message: str
    session_id: str = "default"

class ChatResponse(BaseModel):
    answer: str
    sources: list[str] = []
    tokens_used: int = 0

@app.post("/chat", response_model=ChatResponse)
async def chat(request: ChatRequest):
    """Send a message to the TechNova support agent."""
    result = multi_agent.invoke({
        "messages": [HumanMessage(content=request.message)],
        "next_agent": "", "iteration_count": 0
    })
    answer = extract_final_answer(result)
    return ChatResponse(answer=answer)

# Run: uvicorn main:app --host 0.0.0.0 --port 8000
# Docs auto-generated at http://localhost:8000/docs
```

Streaming: Real-Time Token-by-Token Responses

Users expect word-by-word responses (like ChatGPT). Without streaming: 5-10 second wait. With streaming: first tokens in 200-500ms.

```
from fastapi.responses import StreamingResponse
import json

@app.post("/chat/stream")
async def chat_stream(request: ChatRequest):
    async def event_generator():
        for event in multi_agent.stream(
            {"messages": [HumanMessage(content=request.message)],
             "next_agent": "", "iteration_count": 0},
            stream_mode="updates"
        ):
            for node_name, update in event.items():
                if "messages" in update:
                    last = update["messages"][-1]
                    if hasattr(last, "content") and last.content:
                        data = json.dumps({"node": node_name, "content": last.content})
                        yield f"data: {data}\n\n"  # Server-Sent Events format
        yield f"data: {json.dumps({'type': 'done'})}\n\n"

    return StreamingResponse(event_generator(), media_type="text/event-stream")
```

Frontend consumes with EventSource API (JS) or fetch + ReadableStream. Same protocol ChatGPT uses.

Webhooks: Connecting to Chat Platforms

```python
# Webhook pattern: Slack sends POST to your URL, you respond
# [Slack] --POST /webhook/slack--> [FastAPI] --invoke--> [Agent] --> [Slack API]

from fastapi import FastAPI, Request
import httpx

@app.post("/webhook/slack")
async def slack_webhook(request: Request):
    body = await request.json()

    # Slack verification challenge
    if "challenge" in body:
        return {"challenge": body["challenge"]}

    event = body.get("event", {})
    user_message = event.get("text", "")
    channel_id = event.get("channel", "")

    # Run agent
    result = multi_agent.invoke({"messages": [HumanMessage(content=user_message)],
                                 "next_agent": "", "iteration_count": 0})
    answer = extract_final_answer(result)

    # Post back to Slack
    async with httpx.AsyncClient() as client:
        await client.post("https://slack.com/api/chat.postMessage",
            headers={"Authorization": f"Bearer {SLACK_BOT_TOKEN}"},
            json={"channel": channel_id, "text": answer})
    return {"ok": True}
```

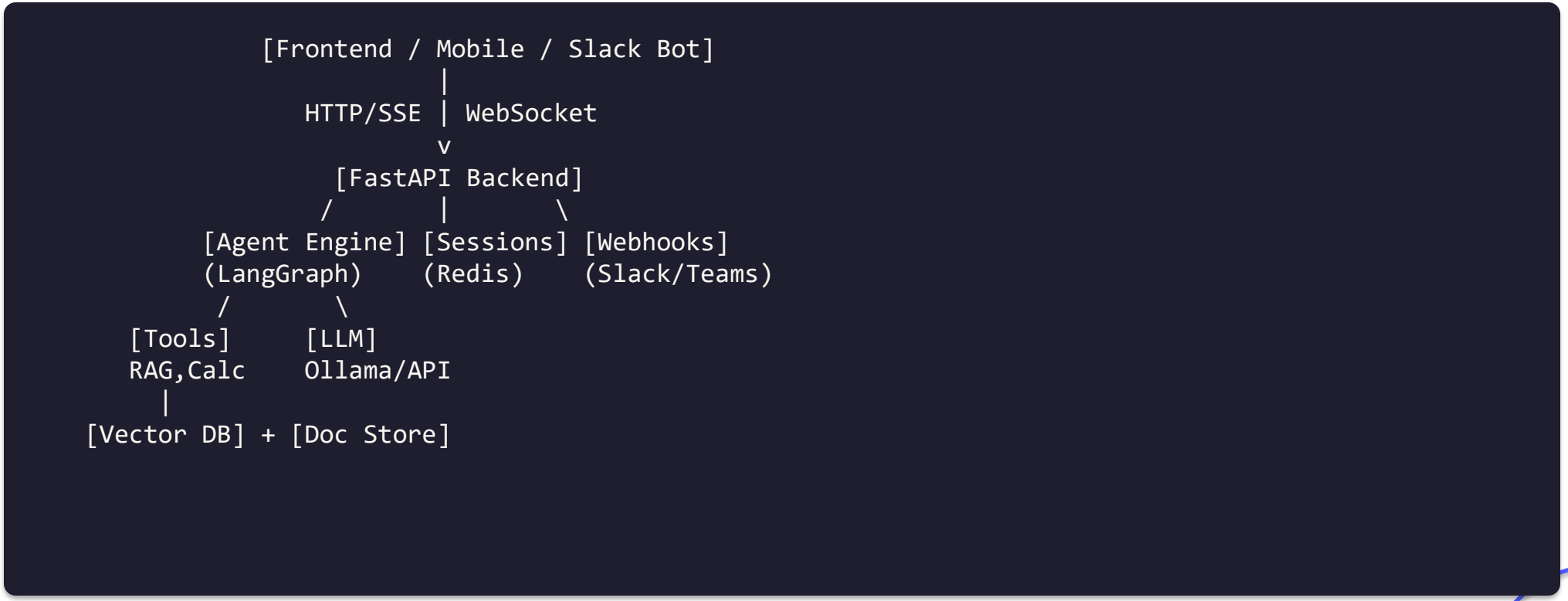
Session Management: Conversation State

```
from collections import defaultdict
import uuid

# Session store (Redis in production)
sessions = defaultdict(list)

@app.post("/chat")
async def chat(request: ChatRequest):
    sid = request.session_id or str(
        uuid.uuid4())
    history = sessions[sid]
    history.append(
        HumanMessage(content=request.message))

    result = multi_agent.invoke({
        "messages": history,
        "next_agent": "",
        "iteration_count": 0})

    sessions[sid] = result["messages"]
    answer = extract_final_answer(result)
    return {"answer": answer,
            "session_id": sid}
```

- **Production storage options:**
- **Redis:** Fast, in-memory, TTL for auto-expiration. Best for chat.
- **PostgreSQL:** Persistent, queryable, good for conversation analytics.
- **LangGraph Checkpointers:** Built-in support for SQLite, PostgreSQL, Redis.

Critical: set TTL (time-to-live) on sessions. 24-hour TTL = conversations expire after inactivity. Without TTL, memory grows forever.

Complete API Architecture



This architecture works for any LLM application. The frontend communicates with your API via HTTP/SSE(Server-sent-events). The API handles agent execution, session management, and webhook integrations.

API Best Practices for LLM Applications

- **Always use async:** LLM calls take seconds. Synchronous code blocks the server. Use async def + await for all endpoints.
- **Request timeouts:** Set max 30-60 seconds for agent execution. Return partial response or error if exceeded.
- **Structured logging:** Log every request/response as JSON. You need this for debugging, monitoring, and evaluation.
- **Rate limiting:** Protect against abuse. Limit requests per user per minute. Use slowapi library or Redis counters.
- **Health checks:** GET /health returns 200 if running. Load balancers and monitoring use this.
- **API versioning:** POST /v1/chat allows breaking changes in /v2/chat without disrupting existing clients.
- **Graceful errors:** Never return stack traces to users. Clean error messages with HTTP status codes.
- **Documentation:** FastAPI auto-generates OpenAPI docs. Add descriptions to every endpoint and field.

Part 4: Inference Optimization

Making models faster, cheaper, and smaller without losing quality.

Why Inference Optimization Matters

Inference is the main cost driver. The math:

Scenario	Daily Queries	Monthly Cost
Low traffic	1,000/day	~$300
Medium traffic	10,000/day	~$3,000
High traffic	100,000/day	~$30,000

Component	Latency Issue	User Impact
LLM generation	2-10 seconds	Users leave after 3s
RAG retrieval	200-500ms	Acceptable
Voice pipeline	1.5-7 seconds	Not conversational

Three categories: making the model smaller (quantization), serving more efficiently (vLLM, batching), reducing redundant work (caching). Based on GPT-4o pricing at ~$2.50/1M input, ~$10/1M output tokens, ~2000 input + 500 output tokens per query.

Quantization: Making Models Smaller

Reduce numerical precision of weights. 32-bit -> 4-bit. Model is 4-8x smaller and faster with small accuracy loss.

Precision	Bits/weight	Size (7B model)	VRAM	Accuracy Loss	Speed Gain
FP32	32	~28 GB	~28 GB	Baseline	1x
FP16/BF16	16	~14 GB	~14 GB	Minimal	~2x
INT8 (Q8)	8	~7 GB	~7 GB	Minimal	~2-3x
INT4 (Q4)	4	~3.5 GB	~4 GB	Small	~3-4x
INT2 (Q2)	2	~1.75 GB	~2 GB	Noticeable	~4-5x

- **GGUF:** Used by llama.cpp and Ollama. CPU-friendly. Variants: Q4_K_M (good balance), Q5_K_M (better quality), Q8_0 (near-lossless). This is what you use with Ollama.
- **GPTQ:** GPU-focused 4-bit. Used by many HuggingFace models. Requires GPU for inference.
- **AWQ:** Activation-Aware Weight Quantization. Newer, better quality than GPTQ at same bits. Gaining popularity.

vLLM: High-Throughput Model Serving

vLLM (UC Berkeley, 2023): open-source fast LLM serving. Standard for self-hosted production. Key innovation: PagedAttention (manages memory like an OS, 6x faster than naive HuggingFace).

Framework	Tokens/sec	Relative
HuggingFace	~800	1x
TGI (HuggingFace)	~2,400	3x
vLLM	~4,800	6x

```
# Serve with OpenAI-compatible API
python -m vllm.entrypoints.openai.api_server \
    --model meta-llama/Llama-3-8B-Instruct \
    --port 8000

# Use with OpenAI client:
client = OpenAI(base_url="http://localhost:8000/v1")
```

- **Features:** Continuous batching, OpenAI-compatible API, tensor parallelism (multi-GPU), GPTQ/AWQ/GGUF support, prefix caching.
- **Reference:** https://docs.vllm.ai/

Batching and Caching: Reducing Redundant Work

```python
# Response caching
import hashlib
cache = {}

def cached_call(question: str):
    key = hashlib.md5(
        question.lower().strip().encode()
    ).hexdigest()
    if key in cache:
        return cache[key]  # Hit!
    result = run_agent(question)
    cache[key] = result
    return result
```

- **Continuous batching:** Process multiple requests simultaneously on GPU. vLLM/TGI handle automatically. 3-8x throughput.
- **Semantic caching:** Embed the question, check if similar Q was answered (cosine > 0.95). Return cached answer. GPTCache library.
- **KV-cache / prefix caching:** Reuse key-value pairs for shared system prompts across requests. vLLM prefix caching.
- **OpenAI Batch API:** 50% cheaper for batch jobs with 24h turnaround. For non-real-time tasks.

Part 5: Hardware Fundamentals

GPU vs CPU, VRAM requirements, and why NVIDIA dominates. What you need to know for deployment decisions.

GPU vs. CPU for AI Inference

Factor	CPU	GPU
Cores	4-64 (powerful, general)	1,000-16,000 (simple, parallel)
LLM speed	~1-5 tokens/sec (7B)	~30-100 tokens/sec (7B)
Memory	16-128 GB (system RAM)	8-80 GB (dedicated VRAM)
Cost	$0 (have it already)	$500-$40,000+ per GPU
Best for	Dev, low traffic, tiny models	Any production workload

- **Apple Silicon exception:** M1-M4 chips: unified memory (CPU+GPU share RAM). 32 GB MacBook runs models needing 32 GB GPU. ~60-70% NVIDIA performance. Why Ollama works great on Mac.
- **When CPU is OK:** Development, < 10 queries/hour, models < 3B params, batch processing where latency is fine.

LLM inference is matrix multiplication, which is massively parallel. GPUs have thousands of cores for parallel work. CPUs have a few powerful cores for serial work. This is why GPUs are 10-50x faster for LLMs.

The NVIDIA Stack: GPU Lineup for Inference

GPU	VRAM	FP16 TFLOPS
RTX 4060	8 GB	15.1
RTX 4070 Ti	12 GB	22.6
RTX 4090	24 GB	82.6
A100 (40GB)	40 GB	312
A100 (80GB)	80 GB	312
H100	80 GB	989
H200	141 GB	989

VRAM is the bottleneck. Rough formula: VRAM (GB) = Params (B) x Bytes_per_param x 1.2. FP16: 2 bytes (7B = 16.8 GB). Q4: 0.5 bytes (7B = 4.2 GB). Q8: 1 byte (7B = 8.4 GB).

NVIDIA dominance comes from CUDA ecosystem. Every ML framework is CUDA-optimized. AMD ROCm and Intel oneAPI are catching up but ecosystem gap is still significant.

Cloud GPU Pricing

Provider	GPU	VRAM	On-Demand $/hr	Spot $/hr
AWS	A10G	24 GB	~$1.00	~$0.40
AWS	A100 (40GB)	40 GB	~$3.80	~$1.50
AWS	H100	80 GB	~$12.00	~$4.50
Azure	A100 (80GB)	80 GB	~$3.40	~$1.40
GCP	A100 (80GB)	80 GB	~$3.60	~$1.10
Lambda Labs	A100 (80GB)	80 GB	~$1.10	N/A
RunPod	A100 (80GB)	80 GB	~$1.64	~$0.74

Part 6: Fine-Tuning Decision Framework

When to prompt engineer, when to RAG, when to fine-tune, and when to combine all three.

The Model Adaptation Spectrum

```
Prompt Engineering  ->  RAG  ->  Fine-Tuning  ->  Pretraining from Scratch

Cost:    $0             $-$$       $$-$$$          $$$$$$
Time:    Minutes        Hours      Days-Weeks      Months
Data:    0 examples     Documents  100s-1000s      Billions of tokens
Control: Low            Medium     High            Total
```

- **Prompt engineering:** Better instructions. Zero cost, instant. Handles 60-70% of use cases. (Session 7)
- **RAG:** External knowledge access. Solves "model doesn't know my data." (Sessions 8-10)
- **Fine-tuning:** Update model weights on your data. Changes behavior, style, domain expertise. Needs labeled data.
- **Pretraining:** Train from scratch. Only large labs (OpenAI, Google, Meta). Not practical for most orgs.
- **Key insight:** These are complementary, not competing. You might prompt-engineer a fine-tuned model that uses RAG.

When Fine-Tuning IS the Right Choice

- **Changing output format/style:** Need consistent JSON schema, specific writing style, domain-specific language. Prompting helps but fine-tuning makes it reliable.
- **Teaching domain vocabulary:** Medical, legal, financial, technical domains. Model needs specific terminology consistently.
- **Reducing latency and cost:** Fine-tuned small model (7B) can match prompted large model (70B) on specific tasks. Smaller = faster + cheaper.
- **Consistent behavior:** Need the same behavior across thousands of requests. Fine-tuning bakes it into weights. Prompting can be fragile.
- **Classification tasks:** Sentiment, intent, entity extraction, moderation. Fine-tuned models significantly outperform prompted models.
- **Following complex instructions:** Very long system prompts get partially ignored. Fine-tuning compresses instructions into weights.

When Fine-Tuning IS the WRONG Choice

- **Adding knowledge:** Fine-tuning is BAD for injecting facts. Model memorizes but cannot distinguish fine-tuned knowledge from hallucination. Use RAG for knowledge.
- **You have less than 100 examples:** Too few examples = overfitting. Model memorizes training set instead of learning patterns. Minimum: 100-500 high-quality examples.
- **Task changes frequently:** Fine-tuning takes hours/days and costs money. If requirements change weekly, you re-fine-tune constantly. Prompting/RAG adapts instantly.
- **Prompt engineering already works:** If good prompt + few-shot gives acceptable results, do not fine-tune. Unnecessary complexity.
- **You cannot evaluate properly:** Without a good eval dataset and metrics, you cannot tell if fine-tuning helped. You might make things worse.

Fine-Tuning Decision Tree

```
Does the model need specific facts/data?
    YES -> Use RAG (not fine-tuning)
    NO  -> Continue...

Does prompt engineering give acceptable results?
    YES -> Stop here. Use prompt engineering.
    NO  -> Continue...

Is the issue about output FORMAT or STYLE?
    YES -> Fine-tuning is likely right

Is it a CLASSIFICATION task (intent, sentiment, entity)?
    YES -> Fine-tuning is the right choice

Do you have 100+ high-quality labeled examples?
    YES -> Fine-tuning is viable
    NO  -> Collect more data, or use few-shot prompting
```

In practice: Prompt + RAG (80% of use cases). Prompt + RAG + fine-tuned (15%). Fine-tuning alone (5%).

Fine-Tuning: Practical Options

Method	What It Is	Cost	Data Needed	Providers
Full fine-tuning	Update all weights	$$$	1000+ examples	Your GPUs, cloud
LoRA	Small adapter matrices, freeze base	$	100-500	HuggingFace, Unsloth
QLoRA	LoRA on 4-bit quantized model	$	100-500	HuggingFace, Unsloth
OpenAI fine-tuning	Managed GPT tuning	$$	50-100+	OpenAI API
Vertex AI tuning	Managed Gemini tuning	$$	100+	GCP
Bedrock fine-tuning	Managed select models	$$	100+	AWS

LoRA is the standard for open models. Adds small trainable matrices to frozen base. 10-100x cheaper than full fine-tuning. Adapter is 10-100 MB vs 14 GB for full model.

References: LoRA paper: https://arxiv.org/abs/2106.09685, Unsloth (fast LoRA): https://github.com/unslothai/unsloth

Part 7: Security, Cost, and Scalability

The non-glamorous but critical aspects that determine whether your system survives in production.

Security for LLM Applications

- **API key management:** Never hardcode keys. Use environment variables (.env + python-dotenv), cloud secrets managers (AWS Secrets Manager, Azure Key Vault, GCP Secret Manager), separate keys per environment.

- **Data privacy:** What data goes to external APIs? OpenAI API: data NOT used for training. Azure OpenAI: stays in your tenant. Local models: never leaves your machine. Check GDPR/HIPAA/PCI-DSS requirements.

```
# BAD - key in code
client = OpenAI(api_key="sk-abc123")

# GOOD - env variable
import os
client = OpenAI(
    api_key=os.environ["OPENAI_API_KEY"])

# BEST - .env file + python-dotenv
from dotenv import load_dotenv
load_dotenv()
key = os.getenv("OPENAI_API_KEY")
```

- **Prompt injection defense (layered):** Input validation (regex) + LLM-based classification (separate model) + output validation (check for leaked prompts, PII) + least privilege (agent tools have minimal permissions).

OWASP Top 10 for LLM Applications (2025)

#	Risk	What It Is	Your Defense
1	Prompt Injection	Manipulate LLM via crafted input	Input guardrails, separate data/instruction
2	Sensitive Info Disclosure	LLM leaks training/user data	Output filtering, PII redaction
3	Supply Chain Vulnerabilities	Compromised models/plugins	Verify sources, pin versions
4	Data/Model Poisoning	Malicious training data	Data validation, trusted sources
5	Improper Output Handling	LLM output causes SQLi	Sanitize output before downstream use
6	Excessive Agency	Agent has too many permissions	Least privilege, HITL for destructive ops
7	System Prompt Leakage	Attacker extracts system prompt	No sensitive info in prompts, detect extraction
8	Vector/Embedding Weaknesses	Manipulation of vector DBs	Access controls, validate data before indexing
9	Misinformation	Plausible but false content	RAG grounding, faithfulness eval, citations
10	Unbounded Consumption	DoS via excessive LLM usage	Rate limiting, iteration caps, cost budgets

Reference: https://genai.owasp.org/

**The Open Worldwide Application Security Project (OWASP) is a nonprofit foundation dedicated to improving software security through free, open-source resources, tools, and community-driven documentation

Cost Optimization Strategies

- **1. Model routing (most impactful):** Cheap model for easy requests, expensive for hard ones. GPT-4o-mini: $0.15/1M input. GPT-4o: $2.50/1M. 17x difference.
- **2. Prompt optimization:** Remove unnecessary instructions. 500 vs 2000-token system prompt = 4x cost difference on every request.
- **3. Response caching:** Similar questions get cached answers. Semantic caching for near-matches.
- **4. Token-aware design:** Retrieve 3 chunks not 10 unless needed. Set max_tokens on generation. Truncate old conversation history.
- **5. Prompt caching:** Anthropic prompt caching: 90% cheaper for repeated system prompts. vLLM prefix caching for self-hosted.

Cost Tracking and Budget Enforcement

```python
import tiktoken

def estimate_cost(model, input_text, output_text):
    pricing = {
        "gpt-4o": {"in": 2.50, "out": 10.00},
        "gpt-4o-mini": {"in": 0.15, "out": 0.60},
    }  # per 1M tokens
    enc = tiktoken.encoding_for_model(model)
    in_tok = len(enc.encode(input_text))
    out_tok = len(enc.encode(output_text))
    rates = pricing[model]
    cost = (in_tok * rates["in"] +
            out_tok * rates["out"]) / 1_000_000
    return cost, in_tok, out_tok
```

```python
# Per-user daily budget
user_budgets = {}
MAX_DAILY = 1.00  # $1/user/day

async def check_budget(user_id, est_cost):
    today = datetime.now().strftime("%Y-%m-%d")
    key = f"{user_id}:{today}"
    current = user_budgets.get(key, 0)
    if current + est_cost > MAX_DAILY:
        return False  # Over budget
    user_budgets[key] = current + est_cost
    return True
```

Monitor cost weekly. A new feature adding 500 tokens/request can double monthly bill. Set alerts for spend thresholds on OpenAI/Anthropic dashboards.

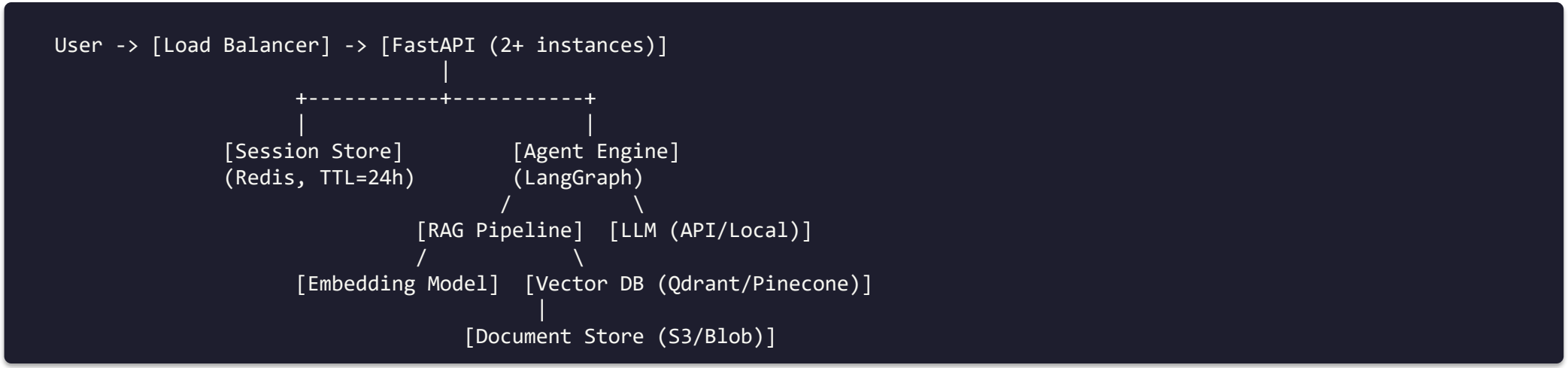
Scalability Patterns for LLM Applications

- **Horizontal scaling:** Multiple FastAPI instances behind a load balancer. Each handles different requests. LLM calls are the bottleneck, not your code.
- **Queue-based architecture:** Requests go to a message queue (Redis Queue, RabbitMQ, SQS). Workers consume from queue. Decouples ingestion from processing. Handles traffic spikes.
- **Async processing:** Non-real-time tasks (indexing, batch eval, reports): return a job ID, let user poll or receive webhook callback.
- **Auto-scaling:** Cloud rules: if GPU utilization > 70%, add instances. If < 20%, remove. Saves cost during low-traffic hours.
- **CDN and edge caching:** FAQ-style static responses cached at CDN. First request hits API, identical subsequent ones served from cache.
- **Connection pooling:** Use PgBouncer or SQLAlchemy pool for database connections. Without pooling, each request opens a new connection (slow, resource-intensive).

Part 8: Architecture Patterns, Pitfalls, Lessons

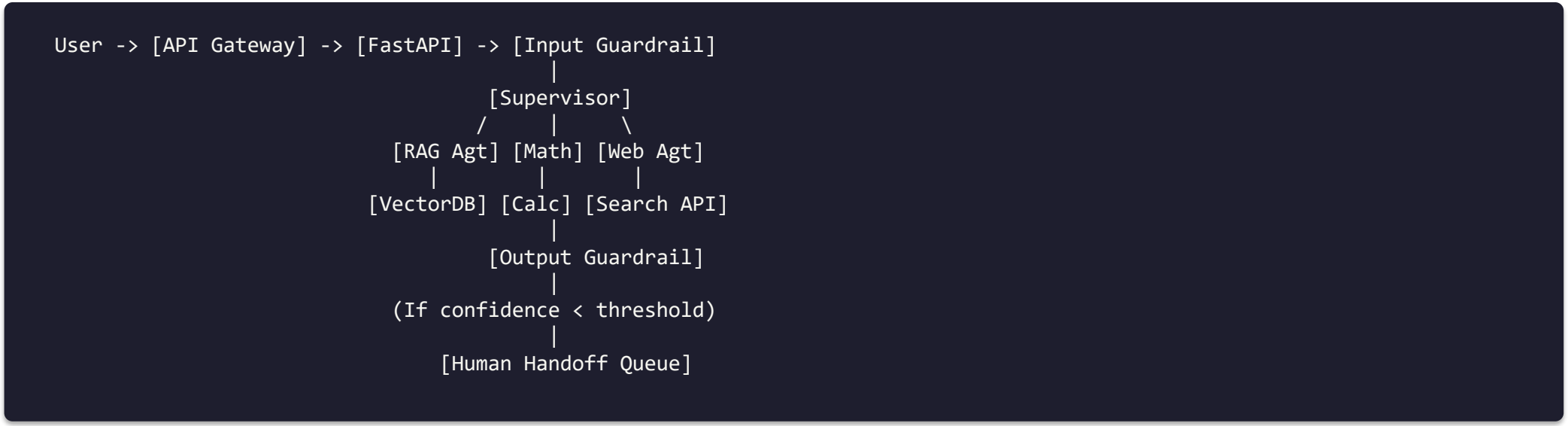
Real-world patterns, common mistakes, and hard-won wisdom from production deployments.

Reference Architecture: Production RAG Chatbot



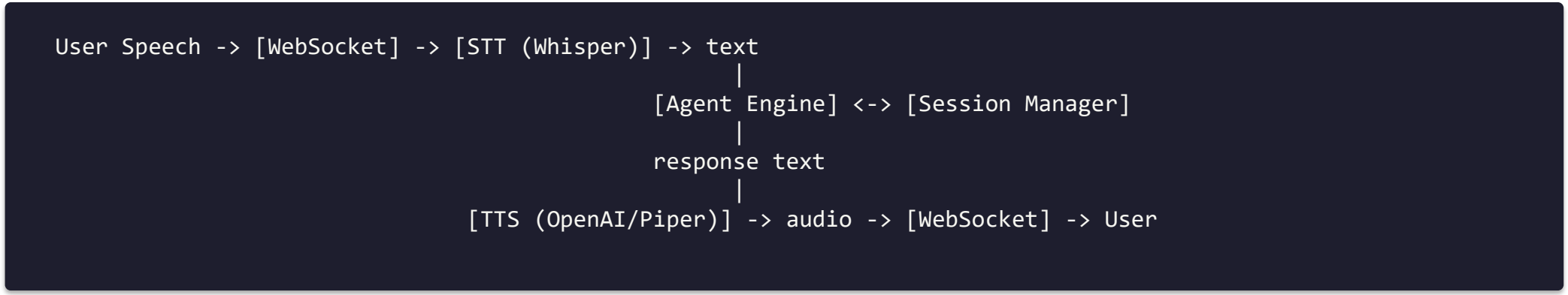
- **Components:** Load balancer (NGINX/ALB), app server (FastAPI 2+), session store (Redis+TTL), agent (LangGraph), RAG (embed+vectorDB), LLM (API or vLLM), doc store (S3), monitoring (LangSmith/Prometheus+Grafana), logging (structured JSON).

Reference Architecture: Multi-Agent Support



- **Additional production components:** Intent classifier for FAQ fast-tracking, confidence scoring on output (low confidence -> human escalation), feedback loop (thumbs up/down -> improve prompts), analytics dashboard (volume, topics, resolution rate, latency, cost).

Reference Architecture: Voice-Enabled Agent



- **Use WebSockets:** Not HTTP. Real-time bidirectional audio streaming requires persistent connections.
- **Voice Activity Detection:** Detect when user stops speaking. Do not cut off mid-sentence.
- **Buffer and batch:** Send audio chunks to STT in batches for efficiency, not one sample at a time.
- **Stream TTS output:** Start playing audio while rest is generating. Reduces perceived latency significantly.
- **Handle interruptions:** If user speaks while agent responds, stop TTS and listen. This is called barge-in.
- **Text fallback:** Always offer text input for noisy environments or accessibility.

The 12 Most Common Production Pitfalls (1-6)

- **1. "It worked in my notebook":** Notebooks hide state. Variables persist between cells. Production: every request starts fresh. Always test in clean environment.
- **2. Ignoring latency:** 10-second response OK in notebook. In production, users leave after 3 seconds. Profile every component.
- **3. No evaluation before deploying:** "Tested 5 questions" is not evaluation. 30+ question test set. Run on every code change. Automate it.
- **4. Unbounded agent loops:** Without iteration limits, agents loop forever burning money. Always set max iterations. Log every step.
- **5. System prompt in user messages:** Users ask "What is your system prompt?" and some models leak it. Treat prompts as sensitive. Add non-disclosure instructions.
- **6. No rate limiting:** One user/bot sends 10,000 requests/minute and bankrupts your API budget. Per-user rate limits from day one.

The 12 Most Common Production Pitfalls (7-12)

- **7. Hardcoded API keys:** Happens to everyone once. Key ends up on GitHub. Environment variables and secrets managers. Always.
- **8. No monitoring or logging:** When something breaks at 2 AM, you need logs. Every request, response, tool call, error. Structured JSON. Centralized.
- **9. Context window overflow:** Long conversations exceed model context. Implement sliding window (last N messages) or summarize older messages.
- **10. Trusting LLM output as code/SQL:** Agent generates SQL that gets executed? It WILL eventually generate DROP TABLE. Validate, sandbox, limit permissions.
- **11. Not handling API outages:** OpenAI, Anthropic, Google all have outages. Retry with backoff, fallback to different model, return "temporarily unavailable."
- **12. Over-engineering on day one:** You do not need Kubernetes and multi-region on day one. Start simple. Add complexity when metrics show you need it.

Lessons from Industry: What Works

- **Start with the simplest architecture that works.** Single FastAPI, Qdrant, API. Add complexity only when metrics show you need it.
- **Evaluation is your compass.** Every decision (chunk size, model, prompt) measured against test set. Intuition is unreliable with LLMs.
- **Prompt engineering gets you 80%.** Invest heavily in prompt design before fine-tuning. Most teams fine-tune too early.
- **RAG quality is bounded by retrieval quality.** Best LLM in the world cannot fix bad context. Invest in retrieval first.
- **Users forgive slow, not wrong.** 5-second correct response builds more trust than 1-second wrong response 20% of the time.
- **Build feedback loops from day one.** Thumbs up/down on responses. Review low-rated weekly. Best source of eval data.
- **Hardest part is data pipeline, not AI.** Parsing, cleaning, chunking, indexing takes more engineering than building the agent.
- **Monitor cost weekly.** LLM costs creep up. A feature adding 500 tokens/request can double your monthly bill.

Production Readiness Checklist

Category	Item
Code	No hardcoded API keys
Code	Async handlers for all LLM calls
Code	Error handling with graceful fallbacks
Code	Structured request/response logging
Security	Keys in secrets manager or env vars
Security	Input guardrails (prompt injection)
Security	Output guardrails (PII, sensitive data)
Security	Per-user rate limiting
Performance	Response timeout (30-60s max)
Performance	Agent iteration limit

Category	Item
Performance	Context window management
Evaluation	Test set with 30+ questions
Evaluation	Retrieval metrics computed
Evaluation	Generation metrics computed
Monitoring	Health check endpoint
Monitoring	Cost tracking and alerting
Monitoring	Error alerting
Operations	requirements.txt / pyproject.toml
Operations	README with setup instructions
Operations	Environment-specific configuration

The Complete AI Engineer Stack

```
SESSIONS 4-6: FOUNDATIONS
  Classical NLP -> Deep Learning -> Transformers

SESSION 7: LLM USAGE
  Local models (Ollama) + API + Prompting + LangChain

SESSION 8: RAG
  Chunking + Embeddings + Vector DBs + Retrieval + Generation

SESSIONS 9-10: AGENTS
  ReAct + Tool Calling + LangGraph + Multi-Agent + MCP + Guardrails

SESSION 11: MULTIMODAL + EVALUATION
  Voice (Whisper + TTS) + Vision (VLMs) + RAGAS + LLM-as-Judge

SESSION 12: PRODUCTION
  Cloud + APIs + Optimization + Security + Architecture + Lessons
```

What's Next: DevOps + Final Project

Next session: A DevOps engineer will teach Docker, containerization, and deployment. You will containerize the application you build in your final project.

Your final project combines multiple components from across the course into a complete, evaluated, documented application.

The project is your portfolio piece. It demonstrates to potential employers that you can build real AI applications, not just run notebooks.

Project topics and requirements will be defined in a separate document. Start thinking about what you want to build.

inmind.academy
by inmind.ai

THANK YOU

Questions?

ADDRESS

Mkalles, main road, bld. 757, Lebanon

CONTACT

Phone +961 70 478 758
Email inmind.academy@inmind.ai
Website www.inmind.academy

SOCIAL MEDIA